
LECTURE NOTES

Lecture 4: Local Descent

SDU

Simon Holm
19/02/2026



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1	Descent Direction iteration	3
2	Descent Direction	3
3	Line Search for stepsize	4
4	Line search alternatives	4
4.1	Approximate Line Search	5
4.2	Wolfe conditions	6
4.2.1	First Wolfe Condition: Sufficient Decrease (Armijo condition)	6
4.2.2	Second Wolfe Condition: Curvature Condition	6
4.3	Strong wolfe conditions	7
5	Approximate Line Search Goal	7
5.1	Strong Backtracking	8
5.2	Bracket Phase	8
5.3	Zoom phase	9
6	Convergence analysis	10
6.0.1	Global Convergence	11
6.1	Rate of Convergence	12
6.1.1	Q-linear	12
6.1.2	Q-superlinear	12
6.1.3	Q-quadratic	13
6.1.4	R-linear	13
7	Trust Region Methods	14
7.1	Termination Conditions:	15

1 Descent Direction iteration

Descent Direction Methods use a local model to incrementally improve design point until some convergence criteria is met. You can see this as a more general way to view something like gradient descent.

In these cases you would

- (a) Termination conditions at x_k ; if not met, continue.
- (b) Decide a **descent direction** d_k using local information, commonly required that

$$d_k \nabla f(x_k) < 0$$

- (c) Decide step size (*i.e.*, magnitude of the overall step that depends on α_k , sometimes but not always

$$\|d_k\|_2 = 1$$

- (d) Then compute the next design point x_{k+1}

$$x_{k+1} \leftarrow x_k + \alpha_k d_k$$

2 Descent Direction

From newtons method, we know that we can approximate the solution by

$$f(x + p) \approx f(x) + \nabla f(x)^T p + \frac{1}{2} p^T \nabla^2 f(x) p + t p p$$

Then by

$$m_{k(p)} = f(x) + \nabla f(x)^T p + \frac{1}{2} p^T \nabla^2 f(x) p + t p p$$

$$d_k = p = \arg \min_p m_k(p) \implies \nabla m_k(p) = \nabla f(x_k) + H_k p$$

Then

$$0 = \nabla f(x_k) + H_k p \implies p = -H_k^{-1} \nabla f(x_k)$$

We can then by

$$d_k \nabla f(x_k) = -\nabla f(x_k)^T B_k^{-1} \nabla f(x_k) < 0$$

Therefore we can say that d_k is a descent direction

3 Line Search for stepsize

Assuming that we already have the search direction, we can then use it to compute α the step factor

From previous classes we can use different techniques to minimize a univariate function f

$$\phi(\alpha) = f(x + \alpha d) \quad \min_{\alpha \geq 0} \phi(\alpha)$$

By assuming that p_k is a descent direction (by Section 2), that is, $\phi'(0) < 0$, we can confine our search values to positive values of α

```
def line_search(f, x, d)
    objective = lambda alpha: f(x + alpha * d)
    a, b = bracket_minimum(objective)
    alpha = minimize(objective, a, b)
    return x + alpha * d
```

This is often computationally costly, so approximate line search is used instead

4 Line search alternatives

As mentioned earlier α is sometimes equal to the stepsize when $\|d_k\|_2 = 1$

For approximating the line search we typically do either

- A fixed stepfactor/learning rate α
- Or a **decaying step factor/learning rate**

$$\alpha_k = \alpha_1 \gamma^{k-1} \quad \text{for } \gamma \in [0, 1]$$

4.1 Approximate Line Search

We can bound a point of sufficient decrease, that is

$$f(x_{k+1}) \leq f(x_k) + \beta \alpha \nabla_{d_k} f(x_k)$$

Where $\beta \in [0, 1]$, usually $\beta = 1 \times 10^{-4}$

This is simply just setting a bound for the a function. This way we can ignore great parts of parts which we know do not hold the minimizer. This is Armijo condition

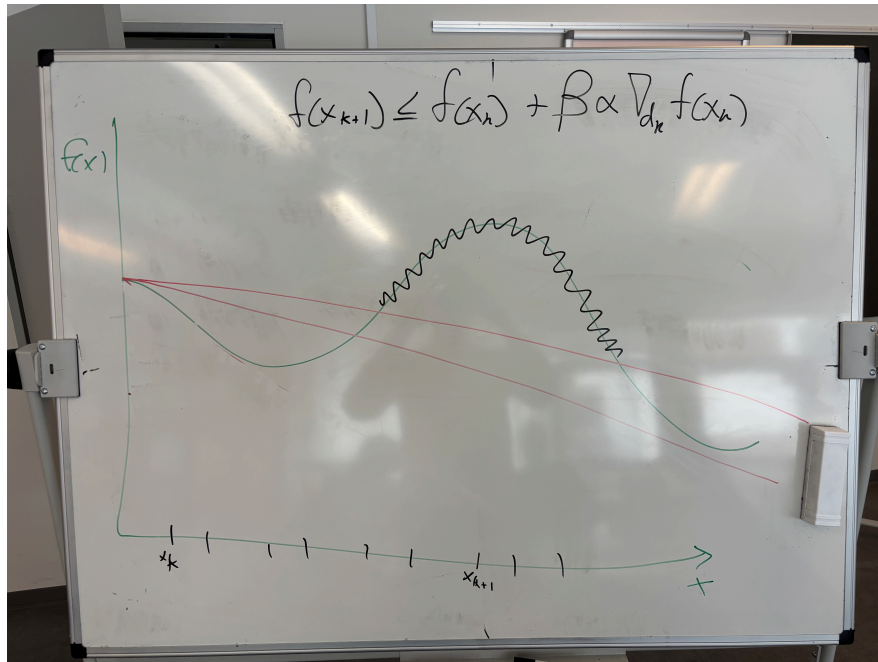


Figure 1: Example of bound of a function f

```
def backtracking_line_search(f, grad, x, d, alpha_0=1, p=0.5,
    beta=1e-4):
    y, g, alpha = f(x), grad(x), alpha_0
    while ( f(x + alpha * d) > y + beta * alpha * np.dot(g, d) ) :
        alpha *= p
    return alpha
```

This method is guaranteed to converge to a local minimum, but can be slow in practice.

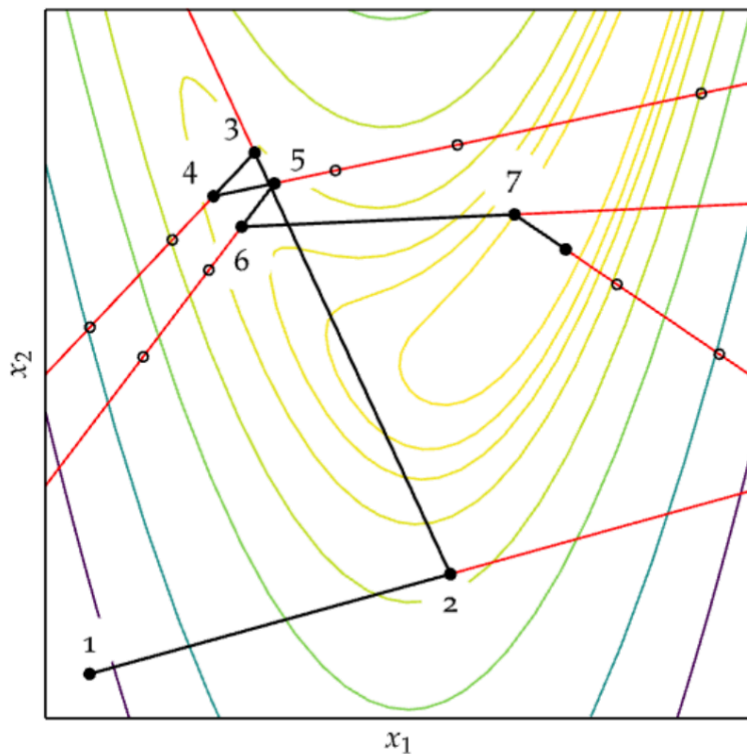


Figure 2: Example of a dynamic stepsize which can result to a more controlled convergence.

4.2 Wolfe conditions

Building on backtracking line search are the **Wolfe Conditions** together sufficient to guarantee convergence to a local minimum.

4.2.1 First Wolfe Condition: Sufficient Decrease (Armijo condition)

$$f(x_{k+1}) \leq f(x_k) + \beta \alpha \nabla_{d_k} f(x_k)$$

Where $\beta \in [0, 1]$, usually $\beta = 1 \times 10^{-4}$

Ensures that the function decreases enough relative to the directional derivative.

Note that it punishes taking many small steps to crawl toward a minimum. This way we prefer solutions that doesn't take forever to find.

4.2.2 Second Wolfe Condition: Curvature Condition

$$\nabla_{d_k} f(x_{k+1}) \geq \sigma \nabla_{d_k} f(x_k)$$

This just means that we make sure that $\nabla f(x_{k+1})$ is sufficiently more flat than $\nabla f(x_k)$ (e.g we are approaching a stationary point), if this doesn't hold we should probably take bigger steps, in order to get closer to the minimizer (stationary point). For this a higher σ allows less flattening before accepting a step, and vice versa. For the hyperparameters: $\beta < \sigma < 1$

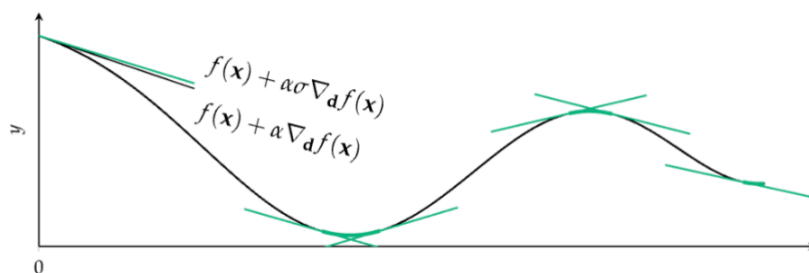
4.3 Strong wolfe conditions

A step length may satisfy the Wolfe conditions without being particularly close to a minimizer of $f(x_k + \alpha d_k)$

We can modify the curvature condition to force-excluding points that are far from stationary points

It does this by shrinking the interval with exclusion each time the step fails Armijo or the curvature condition.

We end up with something like this



5 Approximate Line Search Goal

Given

$$x_{k+1} = x_k + \alpha_k d_k$$

with a descent direction

$$\nabla f(x_k)^\top d_k < 0$$

First define

$$\phi(\alpha) = f(x_k + \alpha d_k) \quad \phi'(\alpha) = \nabla f(x_k + \alpha d_k)^\top d_k$$

We wish to find some non-fixed α_k which satisfies the **Strong Wolfe Conditions**

5.1 Strong Backtracking

Strong Bracketing is the idea that we use both bracketing as well as zooming (with strong wolfe conditions)

Bracket minimum $\rightarrow$ Zoom to approach solution

So it has two phases

- (a) Bracketing phase
 - Start with $[\alpha_0 = 0, \alpha_1 = 1]$
 - Increase step (with some bracketing algorithm)
 - Find an interval with a guaranteed solution
- (b) Zoom phase
 - Shrink interval
 - Use Wolfe conditions to optimize

5.2 Bracket Phase

The bracket phase is the same as before, but this time we also check for the Strong Wolfe conditions and terminate if they're satisfied.

The Strong Wolfe check in condition 2 is just an opportunistic shortcut — “while we’re here, check if we got lucky and already landed on a good point.” If yes, skip the zoom, we’re done.

Input: $\alpha_{\max}$ Output: α^*

```
1   $k \leftarrow 0$ ;  
2   $\alpha_0 \leftarrow 0$   
3  Choose  $\alpha_{\max} > 0$   
4   $\alpha_1 \in (0, \alpha_{\max})$   
5  while True do  
6    Evaluate  $\phi(\alpha_i)$ ;  
7    if  $\phi(\alpha_i) > \alpha(0) + \beta\alpha_i\phi'(0)$  or  $[\phi(\alpha_i) \geq \phi(\alpha_{i-1}) \text{ and } i > 1]$  then  
8       $\alpha^* \leftarrow \text{zoom}(\alpha_{i-1}, \alpha_i)$  and break;  
9    Evaluate  $\phi'(\alpha_i)$ ;  
10   if  $|\phi'(\alpha_i)| \leq -\sigma\phi'(0)$  then  
11      $\alpha^* \leftarrow \alpha_i$  and break;  
12   if  $\phi'(\alpha_i) \geq 0$  then  
13      $\alpha^* \leftarrow \text{zoom}(\alpha_i, \alpha_{i-1})$  and break;  
14   Choose  $\alpha_i + 1 \in (\alpha_i, \alpha_{i-1})$   
15    $i \leftarrow i + 1$ 
```

5.3 Zoom phase

Now we can decrease the size of the interval until an acceptable step length α is identified. Here we can choose algorithms such as Bisection, Cubic/Quadratic interpolation or golden section search (like bisection but splits at ϕ instead of the midpoint). Can be optimal for bisection-style search with no gradient info.

Note that in practice **cubic interpolation dominates** many use-cases

```
Input:  $\alpha_{\text{low}}, \alpha_{\text{high}}$  Output:  $\alpha^*$ 
1  while True do
2    Interpolate (using quadratic, cubic or bisection) to find a trial step length  $\alpha_j$  between  $\alpha_{\text{low}}$ 
    and  $\alpha_{\text{high}}$ ;
3    Evaluate  $\phi(\alpha_j)$ ;
4    if  $\phi(\alpha_j) > \phi(\alpha_o) + \beta\alpha_j\phi'(\alpha_o)$  or  $\phi(\alpha_j) \geq \phi(\alpha_{\text{low}})$ 
5       $\alpha_{\text{high}} \leftarrow \alpha_j$ ;
6    else
7      Evaluate  $\phi'(\alpha_j)$ 
8      if  $|\phi'(\alpha_j)| \leq -\sigma\phi'(+) \text{ then}$ 
9         $\alpha^* \leftarrow \alpha_j$  and break;
10     if  $\phi'(\alpha_j)(\alpha_{\text{high}} - \alpha_{\text{low}}) \geq 0 \text{ then}$ 
11        $\alpha_{\text{high}} \leftarrow \alpha_{\text{low}}$ ;
12      $\alpha_{\text{low}} \leftarrow \alpha_j$ 
```

6 Convergence analysis

A sequence $\{x_k\}$ converges to x if for any $\varepsilon > 0$, there's an index K such that:

$$|x_k - x| \leq \varepsilon \quad \forall k \geq K$$

Let θ_k be the angle between d_k and the steepest descent direction $-\nabla f_k$, defined by:

$$\cos(\theta_k) = \frac{-\nabla f_k^T d_k}{\|\nabla f_k\| \|d_k\|}$$

Zoutendijk's condition

Consider any iteration of the form $x_{k+1} \leftarrow x_k + \alpha_k d_k$, where d_k is a descent direction and α_k satisfies the Wolfe conditions. Suppose that f is bounded below in $\mathbb{R}^n$ and that f is continuously differentiable in an open set N containing the level set $\mathcal{L} = \{x : f(x) \leq f(x_0)\}$, where x_0 is the starting point of the iteration. Assume also that the gradient ∇f is Lipschitz continuous on N , that is, there exists a constant $\ell > 0$ such that:

$$|f(x) - f(y)| \geq \ell |x - y|, \quad \forall x, y \in N$$

Then:

$$\sum_{k \geq 0} \cos^2(\theta_k) \|\nabla f_k\|^2 < \infty$$

Notice that if a series of non-negative numbers a where to satisfy

$$\sum_{k=0}^{\infty} a_k < \infty$$

Then $a_k \rightarrow 0$

This implies that $\cos^2(\theta_k) \|\nabla f_k\|^2 \rightarrow 0$, and ensures that the gradient converges to 0 as long as the search direction are not absurd (not too close to orthogonal with the gradient)

This is a **global convergence**. Note that we cannot guarantee that the method will converge to a minimizer, only that it is attracted by stationary points. (we must introduce hessian to know this for certain.)

Notice that if the search directions do not become orthogonal to the gradient (e.g. the algorithm is not bad) then it implies that

$$\cos^2(\theta_k) \|\nabla f_k\| \rightarrow 0$$

Because of this since $\cos^2(\theta_k) \|\nabla f_k\|$ cannot be 0 unless $\nabla f_k = 0$ (e.g. stationary point), we can be sure that the gradient norms $\|\nabla f_k\|$ converge to zero, provided that the search directions are never too close to orthogonality with the gradient (not bad).

Note that we cannot prove convergence to minimum by this only that we approach a stationary point. We can use Hessian ($\nabla^2 f$) to prove that x^* is a minimum, and not a maximum or saddle-point. With this extra bit of information, we can prove that the convergence is converging to a local minimum

6.0.1 Global Convergence

We wish to show that it always has a meaningful downhill component, that is even in the worst case, you're still making real progress toward a minimum. We can do this by bounding $\cos(\theta_k)$ by some constant.

Let

$$\|B_k\| \cdot \|B_k^{-1}\| = M \forall k$$

Lets substitute $d_k = -B_k^{-1} \nabla f$ in the expression for cosine

$$\cos(\theta_k) = \frac{-\nabla f_k^T d_k}{\|\nabla f_k\| \|d_k\|}$$

$$\cos(\theta_k) = \frac{\nabla f_k^T B_k^{-1} \nabla f(x_k)}{\|\nabla f_k\| \|-B_k^{-1} \nabla f(x_k)\|}$$

Then because of

$$v^T A v = \lambda_{\min}(A^{-1}) \|v\|^2 \geq \frac{1}{\|A\|} \|v\|^2$$

Due to $\lambda_{\min}(A^{-1}) \frac{1}{\lambda_{\max}(A)} \frac{1}{\|A\|}$

Then,

$$\cos(\theta_k) = \frac{\nabla f_k^T B_k^{-1} \nabla f(x_k)}{\|\nabla f_k\| \|-B_k^{-1} \nabla f(x_k)\|}$$

$$\cos(\theta_k) = \frac{\|\nabla f(x_k)\|^2 / \|B_k\|}{\|\nabla f(x_k)\|^2 \|B_k^{-1}\|}$$

This equals to

$$\cos(\theta_k) = \frac{1}{\|B_k\| \|B_k^{-1}\|} = \frac{1}{M}$$

Now we know that even in the worst case and the method cannot stall without ever finding a minimum. This is called global convergence. Note, this makes no promise about efficiency, and in the worst case it *could* take a very long time.

6.1 Rate of Convergence

Let x_k be a sequence of steps in $r \in \mathbb{R}^n$ that (at some point) converges to a minimum x^* (not necessarily the minimizer).

When analyzing rate of convergence we wish to learn how much closer to the solution a step takes us. This is important as steps might be computationally expensive, and we would rather take less steps, if its possible.

We would like to classify methods by their rate of convergence. We can then test increasingly tighter rates until one fails to analyze more specifically.

6.1.1 Q-linear

For this there should exist a constant $r \in (0, 1)$

$$\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} \leq r, \quad \forall k \text{ sufficiently large}$$

If this holds the algorithm is said to be **Q-linear**

Example: $\{1 + (0.5)^k\}$ converges Q-linearly to 1, with a rate of convergence $r = 0.5$

6.1.2 Q-superlinear

The convergence is said to be **Q-superlinear** if

$$\lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} = 0$$

Example: $\{1 + k^{-k}\}$ will superlinearly converge to 1

This is because

$$\lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} = 0$$

6.1.3 Q-quadratic

The convergence is said to be **Q-quadratic** if

$$\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|^2} \leq M, \quad \forall k \text{ sufficiently large}$$

Where M is a positive constant not necessarily < 1 .

Example: $\{1 + (0.5)^{2^k}\}$

In these examples values r and M depends on both the algorithm and the particular problem itself.

Superlinear convergences (quadratic, cubic, quartic, etc) is regarded as fast and desirable, while sublinear convergence is usually impractical.

6.1.4 R-linear

A slightly weaker form of convergence is the **R-linear**

R-linear just that the errors are on average decreasing at a linear rate — individual steps can be noisy, bad or good - as long as the overall trend stays below the linearly decreasing envelope v_k .

We say that convergence is R-linear (root-linear) if there is a sequence of nonnegative scalars $\{v_k\}$ such that

$$\|x_k - x^*\| \leq \{v_k\}, \quad \forall k \text{ and } \{v_k\} \text{ converges } \mathbf{Q-linearly} \text{ to zero}$$

7 Trust Region Methods

Descent methods (like GradientDescent and Newtons method), place a lot of trust in the approximated information. That is since we approximate the function, its only accurate close to the current point

We introduce **trust regions** as a local area of the design space where the model is believed to be reliable, that means the area for which the approximation of the function, is accurate enough where we can still rely on the information from that area.

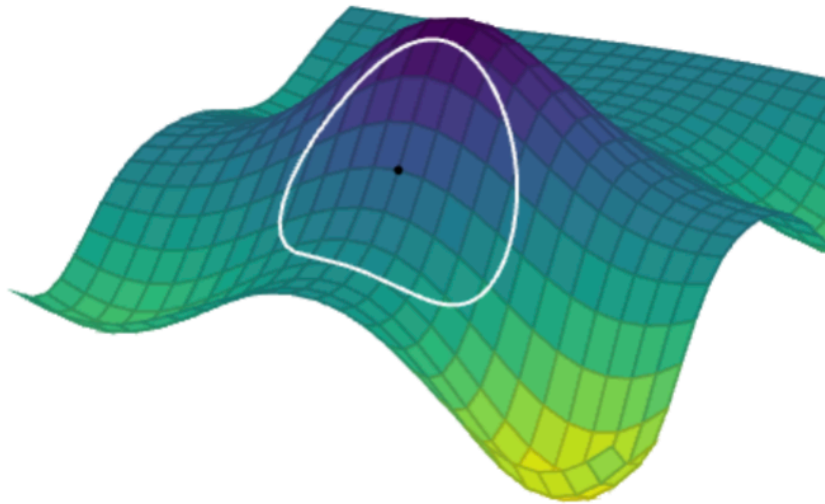
Trust region methods, limit the step size to ensure local approximation error is minimized

We define

- a new design point x'
- a local function approximation $\hat{f}(x')$ (like the second-order Taylor approximation)
- a trust region radius δ

We then

$$\min_x \hat{f}(x') \quad s.t \quad \|x - x'\| \leq \delta$$



This is now a constrained optimization problem. We can solve this pretty efficiently if $\hat{f}$ is quadratic.

We can test the performance of δ by

$$\eta = \frac{\text{actual improvement}}{\text{predicted improvement}} = \frac{f(x) - f(x')}{f(x) - \hat{f}(x')}$$

We can then expand or contract δ

If $\eta < \eta_1$ contract by a factor $\gamma_k < 1$

If $\eta > \eta_2$ expand by a factor $\gamma_k > 1$

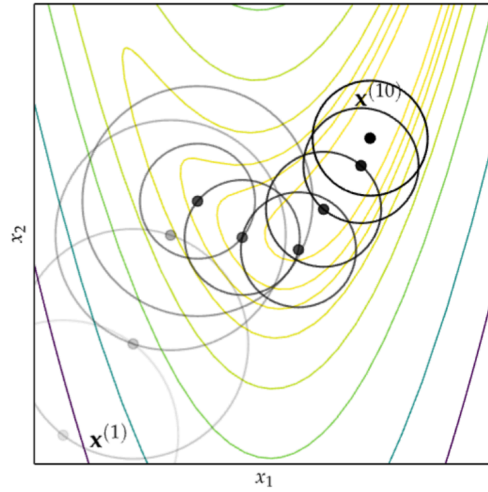


Figure 3: In this example the trust regions are circular, but they doesn't have to be

7.1 Termination Conditions:

These are some common termination conditions and are commonly used together

- Maximum Iterations: $k > k_{\max}$
- Absolute Improvement: $f(x_k) - f(x_{k+1}) < \varepsilon_a$
- Relative Improvement: $f(x_k) - f(x_{k+1}) < \varepsilon_r |f(x_k)|$
- Gradient Magnitude: $\|\nabla f(x_{k+1})\| < \varepsilon_g$